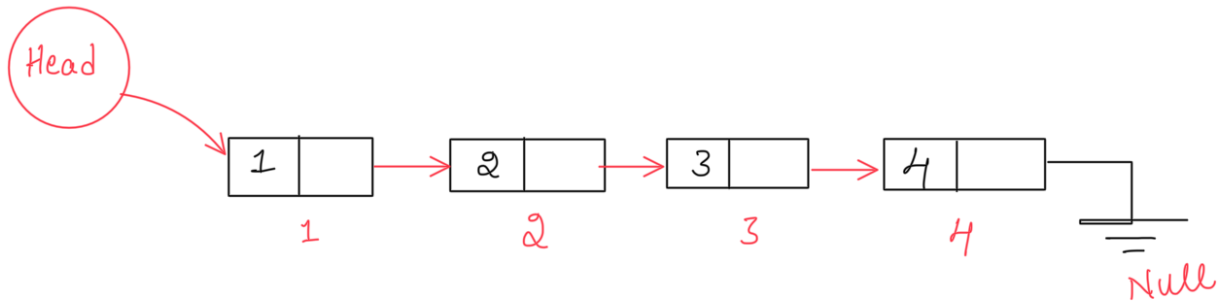
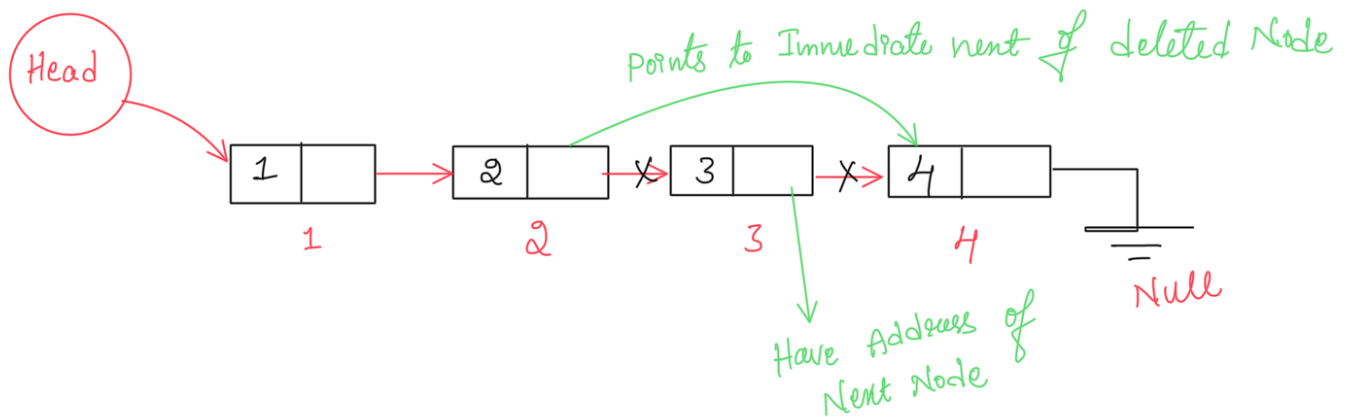


Deleting A Node At Specific position In a LinkedList.

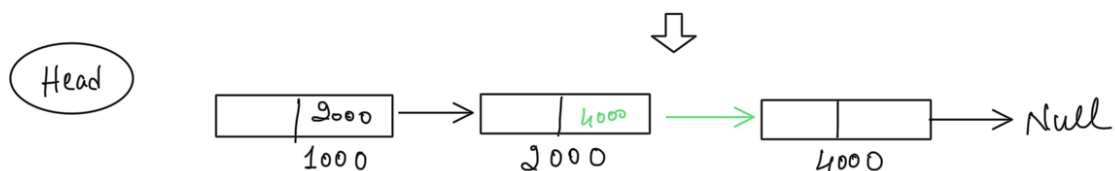
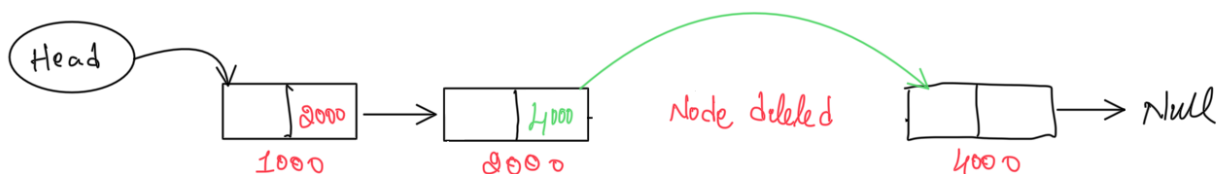
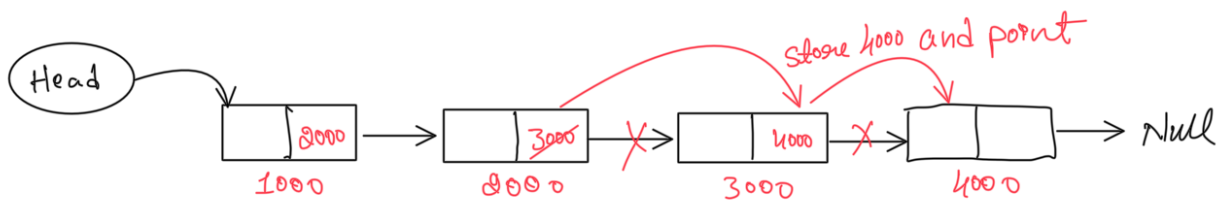
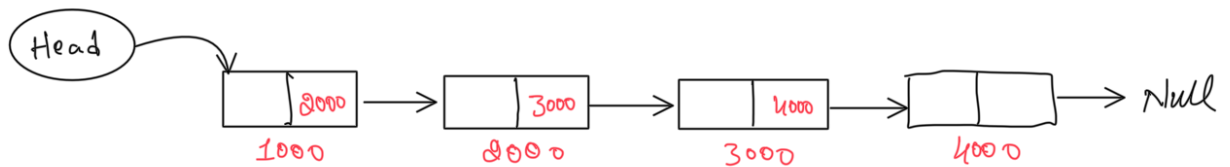
Day 39
18/08/25



Delete a Node At Position 3



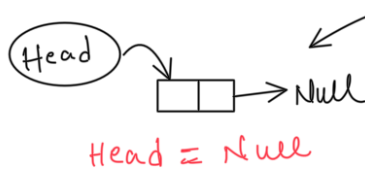
update At one Node previous to the Node which is to be Deleted



Corner Cases

Case ① If list is Empty can't delete item

Case ② If delete position ≤ 0
Invalid delete position



Case ③ If delete position = 1 & the list have only single node then assign null to Head.

Case ④ travel till last previous Node at delete position and adjust links.

✓ Steps with 1-based indexing

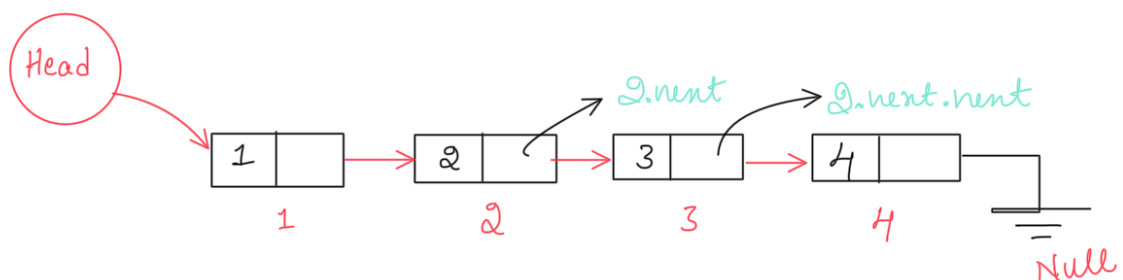
- Position = 1 $\rightarrow$ means delete head node.
- Position > 1 $\rightarrow$ traverse to the node at (pos-1) and adjust links.

⚡ Corner Cases (1-based indexing)

- Empty list $\rightarrow$ head == null.
- Invalid position $\rightarrow$ pos ≤ 0 or pos > size.
- Delete head (pos = 1) $\rightarrow$ head = head.next.
- Delete last node (pos = size) $\rightarrow$ update second last node's next = null.
- Single node list $\rightarrow$ deleting pos = 1 should make head = null.

Case ⑤

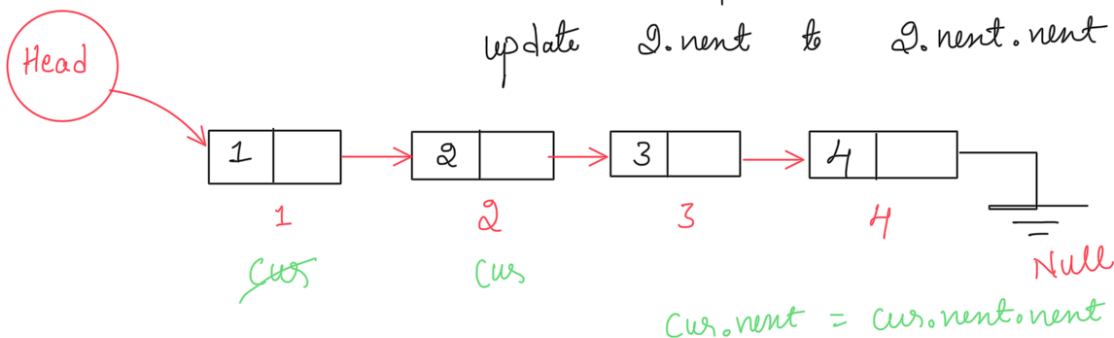
If delete pos = 1
Just make
Head = Head.next.

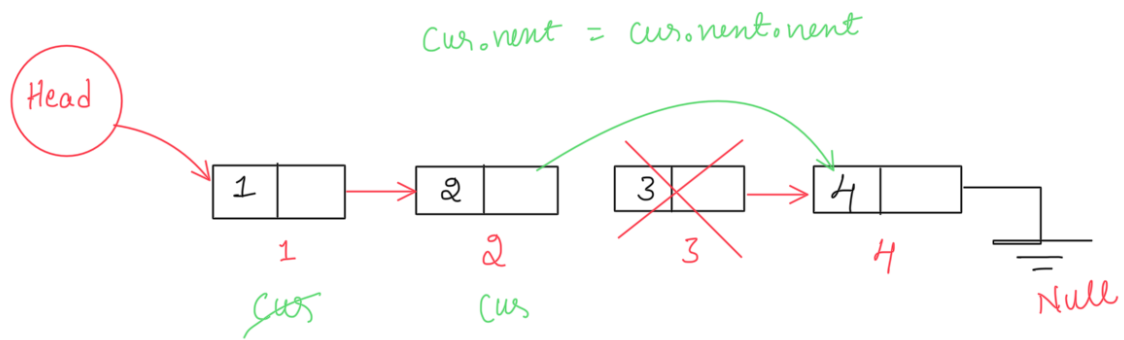


Delete At position 3

travel till 2 position

update 2.next to 2.next.next





NOTE Don't forget to decrement Size after Each deletion.

```

174
175 public void deleteAtPosition(int deleteposition){
176
177     if (this.head == null) {
178         System.out.println("Can't Perform Deletion List is Empty ");
179         return ;
180     }
181
182     if( deleteposition <= 0 || deleteposition > this.size ){
183         System.out.println("Can't Perform Deletion Invalid Delete Position ");
184         return ;
185     }
186
187     //covers Single Node and MultiNode
188     if(deleteposition == 1){
189         this.head = this.head.next;
190         this.size--;
191         return;
192     }
193
194     int position = 1 ;
195
196     Node cur = this.head;
197
198     while (position < deleteposition-1) {
199
200         cur = cur.next;
201
202         if(cur == null){
203             break;
204         }
205
206         position++;
207     }
208
209     if(cur == null){
210         System.out.println("Invalid Delete Position ");
211         return;
212     }
213
214     cur.next = cur.next.next;
215     this.size--;
216
217 }
218
219

```

Head = null
Case

Invalid
delete position

First node deletion

Traversing till last prev
node

optional
BCZ

Invalid
position case

Already Handled

The screenshot shows an IDE with two panels. The left panel displays the source code for `SinglyLinkedList.java`, specifically the `testAllOperations()` method. The right panel shows the execution output, which includes the state of the linked list after various operations like insertion, deletion, and searching.

Textcode

```
5  class SinglyLinkedList {
220 public void testAllOperations(){
265
266     System.out.println("Searching for 20...");
268     this.search(key:20); // should be found if still present
269
270     System.out.println("Searching for 99...");
271     this.search(key:99); // not in list
272
273     System.out.println("\nDeleting at specific positions..");
274     this.deleteAtPosition(deleteposition:1); // delete head
275     this.printAllNodesData();
276     System.out.println("Size: " + size);
277
278     this.deleteAtPosition(deleteposition:3); // delete middle
279     this.printAllNodesData();
280     System.out.println("Size: " + size);
281
282     this.deleteAtPosition(size); // delete last
283     this.printAllNodesData();
284     System.out.println("Size: " + size);
285
286     this.deleteAtPosition(deleteposition:100); // invalid
287     this.printAllNodesData();
288     System.out.println("Size: " + size);
289
290     System.out.println("\nSearching for 20...");
291     this.search(key:20); // should be found if still present
292
293     System.out.println("Searching for 99...");
294     this.search(key:99); // not in list
295 }
296
297 }
```

output

```
Size 5
New Node Object is Created (6)
Inserted Node at end (6)
Head ->1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
Size 6
New Node Object is Created (20)
Head ->1 -> 2 -> 3 -> 20 -> 4 -> 5 -> 6 -> null
New Node Object is Created (-1)
Invalid Position
Head ->-1 -> 1 -> 2 -> 3 -> 20 -> 4 -> 5 -> 6 -> null
Invalid Position
New Node Object is Created (12)
Head ->-1 -> 1 -> 2 -> 3 -> 20 -> 4 -> 5 -> 12 -> 6 -> null
Size 9
New Node Object is Created (55)
Inserted Node at end (55)
Head ->-1 -> 1 -> 2 -> 3 -> 20 -> 4 -> 5 -> 12 -> 6 -> 55 -> null
Size 10
Deleting at front...
Head ->1 -> 2 -> 3 -> 20 -> 4 -> 5 -> 12 -> 6 -> 55 -> null
Size: 9
Deleting at end...
Head ->1 -> 2 -> 3 -> 20 -> 4 -> 5 -> 12 -> 6 -> null
Size: 8
Searching for 20...
Key Found
Searching for 99...
Key is Not Found

Deleting at specific positions...
Head ->2 -> 3 -> 20 -> 4 -> 5 -> 12 -> 6 -> null
Size: 7
Head ->2 -> 3 -> 4 -> 5 -> 12 -> 6 -> null
Size: 6
Head ->2 -> 3 -> 4 -> 5 -> 12 -> null
Size: 5
Can't Perform Deletion Invalid Delete Position
Head ->2 -> 3 -> 4 -> 5 -> 12 -> null
Size: 5

Searching for 20...
Key is Not Found
Searching for 99...
Key is Not Found
(base) kveeresh@Ks-MacBook-Air-2 Computer-Science %
```